Predictive Maintenance for Boiler Temperature and Pressure Monitoring

Submitted by : R.S.N.Shreehitha

Institution : JNTUH

Date : 18/03/2025

Abstract

- This project is designed to create a predictive maintenance system for Industrial boilers through the constant tracking of temperature and pressure readings. Using Machine Learning methods, specially autoencoders.
- This system identifies anomalies that could be signs of impending equipment failures.
- The predictive method minimizes unplanned downtimes as well as the cost of maintenance. The project was able to validate the capability of anomaly detection using simulated data with real-time monitoring functionality.

Table of Contents

- Introduction
- Methodology
- Results
- Conclusion.

- **Problem Statement :**

Industrial Boilers are prone to failures due to excessive temperatures and pressure variations. Predictive maintenance helps in anticipating the failures before they occur , thereby reducing maintenance costs.

- **Objectives :**

- To develop a predictive maintenance system for boiler monitoring.
- To detect anomalies in temperature and pressure data.
- To implement real-time monitoring and alerting.

Introduction

- **Scope :**

The project involves the development of a machine learning model to detect anomalies in boiler temperature and pressure readings. The system will also feature real-time monitoring using web-based dashboards.

Methodology

- Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

Methodology

- Loading and Processing the Data

```
df = pd.read_csv(r"C:\Users\Sai Naga Shreehitha\Downloads\boiler.csv", encoding="latin-1")
df.columns = df.columns.str.strip().str.upper()
if 'DATE' in df.columns:
    df['DATE'] = pd.to_datetime(df['DATE'])
else:
    raise KeyError("DATE column not found in dataset")
df.drop(columns=['UNNAMED: 0'], inplace=True, errors='ignore')
```

- Data Visualization

```
plt.figure(figsize=(12, 5))
sns.lineplot(x=df['DATE'], y=df['TE_8319A.AV_0#'], label='Temperature TE_8319A')
sns.lineplot(x=df['DATE'], y=df['PT_8313A.AV_0#'], label='Pressure PT_8313A')
plt.xlabel('Date')
plt.ylabel('Sensor Readings')
plt.legend()
plt.title('Temperature & Pressure Trends')
plt.show()
```

Methodology

- Feature Engineering

```
rolling_cols = ['TE_8319A.AV_0#', 'PT_8313A.AV_0#', 'TE_8319B.AV_0#', 'PT_8313B.AV_0#']
for col in rolling_cols:
    if col in df.columns:
        df[f'{col}_AVG'] = df[col].rolling(window=5, min_periods=1).mean()
df.fillna(method='bfill', inplace=True)
```

- Features and Target

```
features = ['TE_8319A.AV_0#', 'PT_8313A.AV_0#', 'TE_8319A.AV_0#_AVG', 'PT_8313A.AV_0#_AVG']
X = df[features].copy()
y = df['FAILURE'] if 'FAILURE' in df.columns else None
if y is None:
    raise KeyError("FAILURE column not found in dataset")
```

Methodology

- Standardizing the data and making Predictions

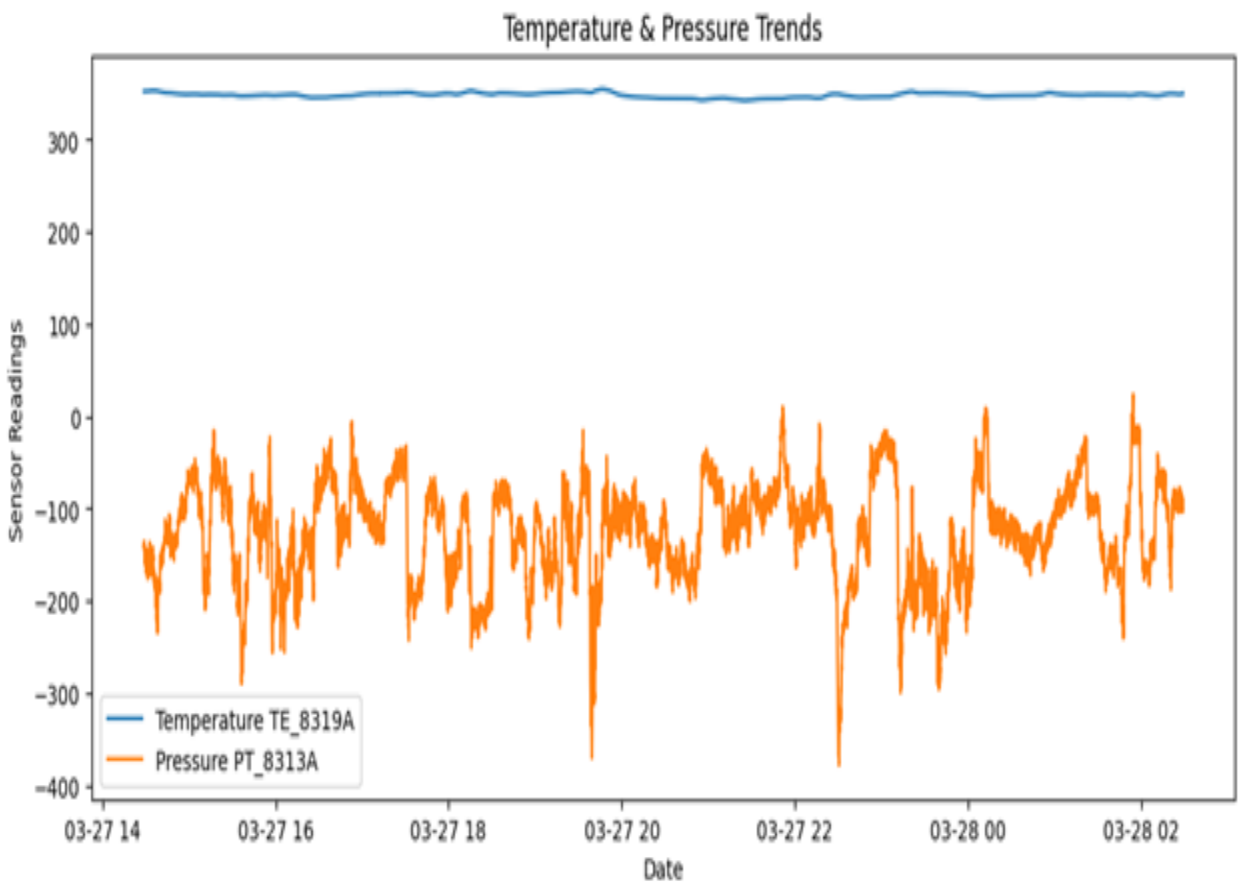
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Methodology

- Feature Visualization

```
feature_importance = pd.Series(model.feature_importances_, index=X.columns)
feature_importance.sort_values().plot(kind='barh', title='Feature Importance', color='teal')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.show()
```

Results


Temperature & Pressure Trends
Sensor Readings
300
200
100
0
−100
−200
−300
−400
03-27 14
03-27 16
03-27 18
03-27 20
03-27 22
03-28 00
03-28 02
Date
Temperature TE_8319A
Pressure PT_8313A

Conclusion

- This project was able to effectively apply autoencoders for predictive maintenance in industrial boilers. The system effectively identified anomalies in temperature and pressure readings, enabling timely maintenance interventions and minimizing unexpected downtime. Real-time monitoring and visualization were realized through a web-based dashboard, improving operational efficiency. In spite of applying simulated data, the performance of the model was encouraging and provides a basis for further work using real data.